

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

**Unit Name: Developing Web Applications - Spring WebFlux MVC
Framework**

Table of Contents

Learning Outcome:	3
1. Spring WebFlux Auto-configuration	4
Leveraging Spring Boot Starters for Spring WebFlux	4
Default Configuration and Customization Options	5
2. HTTP Codecs	6
Working with HTTPMessageReaders for Request Body Handling	6
Using HttpMessageWriters for Response Body Handling	7
Customizing Codecs for Different Media Types and Data Formats	7
3. Static Content in Spring WebFlux	8
Serving Static Resources in a Spring WebFlux Application	8
Configuring Resource Mapping and Handling	9
Caching and Performance Considerations	10
4. Web Filters in Spring WebFlux	10
Creating and Registering Custom Web Filters in Spring WebFlux	11
Configuring Filter Ordering and Execution Chains	12
Handling Common Scenarios with Web Filters	12

UNIT 12: Developing Web Applications - Spring WebFlux MVC Framework

Objectives:

In this Unit you will learn to –

- Understand the concept of auto-configuration and leverage it in Spring WebFlux applications.
- Configure and customize HTTP codecs to handle request and response bodies efficiently.
- Serve static resources and optimize their delivery using caching techniques.
- Implement and configure web filters to intercept and process requests and responses effectively.

Learning Outcome:

Use Spring WebFlux for developing a web application.

12. Developing Web Applications - Spring WebFlux MVC Framework

1. Spring WebFlux Auto-configuration

Auto-configuration in Spring Boot simplifies the configuration process by automatically configuring the Spring WebFlux framework based on the dependencies and settings in the application's classpath. It eliminates the need for explicit configuration and reduces boilerplate code, making it easier to get started with Spring WebFlux.

Auto-configuration is a feature of Spring Boot that automatically configures the application based on its dependencies and the environment. Spring Boot analyzes the classpath and detects the presence of certain libraries and components to determine the necessary configuration. It uses sensible defaults and best practices to provide a fully functional and optimized configuration without requiring explicit configuration from the developer.

Leveraging Spring Boot Starters for Spring WebFlux

Spring Boot starters are a set of convenient dependency descriptors that provide a curated set of dependencies for specific features or frameworks. By including the appropriate Spring Boot starter for Spring WebFlux in the application's build configuration, the necessary dependencies and auto-configuration are automatically included.

To leverage Spring Boot starters for Spring WebFlux, you can add the following dependency to your project:

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-webflux</artifactId>
```

```
</dependency>
```

The `spring-boot-starter-webflux` dependency includes all the necessary dependencies for Spring WebFlux, including the Reactive Streams implementation, the Netty server, and other related components.

Default Configuration and Customization Options

When Spring Boot auto-configures Spring WebFlux, it applies default configurations based on sensible conventions and best practices. However, these defaults can be customized to fit specific application requirements. Here are some default configurations and customization options:

- **Server Configuration:** Spring Boot auto-configures an embedded server (such as Netty or Tomcat) for running the Spring WebFlux application. The server's configuration, including port, SSL, and other properties, can be customized through `application.properties` or `application.yml` files.
 - **Request and Response Handling:** Spring WebFlux auto-configures various components for handling requests and responses, including content negotiation, request parsing, response encoding, and exception handling. You can customize these configurations by registering custom components, implementing custom handlers, or using annotations such as `@ControllerAdvice` and `@ExceptionHandler`.
 - **WebFlux Configuration Properties:** Spring Boot provides a set of properties specifically for configuring Spring WebFlux. These properties can be specified in the `application.properties` or `application.yml` files. For example, you can configure the number of threads in the server thread pool, the maximum size of request and response buffers, and more.
 - **Conditional Auto-configuration:** Spring Boot allows conditional auto-configuration based on certain conditions or the presence of specific dependencies. This allows you to fine-tune the auto-configuration process and override defaults if necessary. Conditional auto-configuration can be achieved by using annotations such as `@ConditionalOnClass`, `@ConditionalOnBean`, or by creating custom condition classes.
 - **Customizing Auto-configuration:** If the default auto-configuration provided by Spring Boot does not meet your application's requirements, you can customize
- Proprietary content. All rights reserved. Unauthorized use or distribution prohibited.

or exclude specific auto-configurations by using the `@EnableAutoConfiguration`, `@SpringBootApplication`, or `@EnableAutoConfiguration(exclude = ...)` annotations.

By leveraging Spring Boot starters and auto-configuration, you can quickly get started with Spring WebFlux without the need for extensive manual configuration. You can customize the auto-configuration to fit your application's requirements by using configuration properties, conditional auto-configuration, or by providing custom components and handlers.

2. HTTP Codecs

In Spring WebFlux, HTTP codecs are used for handling the conversion of request and response bodies to and from Java objects. `HTTPMessageReaders` are responsible for reading the request body and converting it to Java objects, while `HttpMessageWriters` handle the conversion of Java objects to the response body. This allows for flexible and customizable handling of different media types and data formats.

HTTP Codecs in Spring WebFlux are responsible for the conversion of HTTP messages (request and response bodies) to and from Java objects. They provide the necessary logic to handle various media types and data formats, such as JSON, XML, and form data. HTTP codecs work in conjunction with `HTTPMessageReaders` and `HttpMessageWriters` to perform the conversion.

Working with `HTTPMessageReaders` for Request Body Handling

`HTTPMessageReaders` in Spring WebFlux are responsible for reading the request body and converting it to Java objects. They inspect the incoming request, determine the media type of the request body, and use the appropriate decoder to convert the request body to a Java object.

To work with `HTTPMessageReaders`, you can use the `@RequestBody` annotation in your controller methods. The `HTTPMessageReader` associated with the media type

of the request body will automatically be invoked to perform the conversion. You can also customize the `HTTPMessageReaders` by implementing the `HttpMessageReader` interface and registering your custom reader in the application's configuration.

Using `HttpMessageWriters` for Response Body Handling

`HttpMessageWriters` in Spring WebFlux handle the conversion of Java objects to the response body. They inspect the outgoing response, determine the media type to be used in the response, and use the appropriate encoder to convert the Java object to the response body.

To work with `HttpMessageWriters`, you can use the `@ResponseBody` annotation in your controller methods. The `HTTPMessageWriter` associated with the media type of the response body will automatically be invoked to perform the conversion. You can also customize the `HttpMessageWriters` by implementing the `HttpMessageWriter` interface and registering your custom writer in the application's configuration.

Customizing Codecs for Different Media Types and Data Formats

Spring WebFlux provides a range of built-in codecs for common media types and data formats, such as JSON, XML, and form data. However, you can also customize and configure the codecs to handle specific media types and data formats.

To customize the codecs, you can:

- Implement the `HttpMessageReader` or `HttpMessageWriter` interface for your desired media type or data format. This allows you to provide custom logic for reading or writing the data.
- Register your custom codecs in the application's configuration. You can do this by implementing the `WebFluxConfigurer` interface and overriding the `configureHttpMessageCodecs()` method. In this method, you can add your custom codecs to the `Codecs` instance.

For example, to add a custom XML codec, you can create a custom `HttpMessageReader` and `HttpMessageWriter` for XML, and then register them in the

configuration as follows:

```
@Configuration

public class WebFluxConfig implements WebFluxConfigurer {

    @Override

    public void configureHttpMessageCodecs(ServerCodecConfigurer
configurer) {

        configurer.customCodecs().register(new MyXmlHttpMessageReader());

        configurer.customCodecs().register(new MyXmlHttpMessageWriter());

    }

}
```

By customizing the HTTP codecs in Spring WebFlux, you can handle different media types and data formats according to your application's requirements.

3. Static Content in Spring WebFlux

In a Spring WebFlux application, serving static resources such as HTML files, CSS stylesheets, JavaScript files, and images is an essential part of building web applications. Spring WebFlux provides built-in mechanisms to handle and serve static resources efficiently, ensuring optimal performance and delivering a seamless user experience.

Serving Static Resources in a Spring WebFlux Application

To serve static resources in a Spring WebFlux application, you need to place the static content in designated directories. By default, Spring WebFlux looks for static resources in the following directories:

- /static: Static resources placed in this directory will be served as is, without

any modifications.

- /public: Resources in this directory are also served as is, similar to the /static directory.
- /resources: Resources in this directory are served as is, similar to the /static and /public directories.
- /META-INF/resources: Resources in this directory are typically found in JAR files and can be served as static content.

For example, if you have an HTML file named index.html, you can place it in the /static directory, and it will be accessible at the root URL (/index.html) of your application.

Configuring Resource Mapping and Handling

In addition to the default directories, you can also configure custom resource locations and define resource handlers to handle the mapping of URLs to static resources.

To configure resource mapping and handling in Spring WebFlux, you can use the WebFluxConfigurer interface and override the addResourceHandlers() method. Here's an example:

```
@Configuration

public class WebFluxConfig implements WebFluxConfigurer {

    @Override

    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        registry.addResourceHandler("/my-resources/**")

            .addResourceLocations("classpath:/my-static-resources/");

    }

}
```

In the above example, any URL starting with /my-resources/ will be mapped to static resources located in the classpath:/my-static-resources/ directory.

Caching and Performance Considerations

Caching static resources can significantly improve the performance of a web application. When serving static content in a Spring WebFlux application, you can consider the following caching and performance considerations:

1. **HTTP Caching Headers:** Set appropriate caching headers, such as Cache-Control and Expires, to control how the browser and intermediate caches cache the static resources. For example, you can set the Cache-Control header to specify the maximum age of the cached resource.
2. **ETag Support:** ETags are unique identifiers assigned to resources. Implementing ETag support allows the browser to check if the resource has been modified before making a request, reducing unnecessary network traffic. Spring WebFlux provides built-in support for ETags.
3. **Content Compression:** Enable content compression (e.g., gzip) to reduce the size of static resources during transmission, improving the overall performance of the application. Spring WebFlux supports content compression out of the box.
4. **Content Delivery Networks (CDNs):** Consider using a CDN to offload the delivery of static resources. CDNs can help improve the loading speed of static content by serving them from edge servers located closer to the end-users.
5. **Minification and Bundling:** Minify and bundle CSS and JavaScript files to reduce the number of requests made by the client and improve the loading speed. Tools like Webpack or Gulp can be used for this purpose.

4. Web Filters in Spring WebFlux

Web filters in Spring WebFlux are components that intercept and process HTTP requests and responses. They provide a way to modify or inspect the request and response objects, perform pre-processing or post-processing actions, and apply cross-cutting concerns to the web application.

Web filters sit between the web server and the web application and intercept requests and responses. They can modify request parameters, headers, or bodies, as well as perform operations on the response, such as adding headers, modifying content, or handling errors. Web filters are useful for implementing cross-cutting concerns that need to be applied uniformly across multiple endpoints.

Creating and Registering Custom Web Filters in Spring WebFlux

To create a custom web filter in Spring WebFlux, you can implement the `WebFilter` interface. This interface provides two methods: `filter()` and `getOrder()`. The `filter()` method contains the logic to process the request and response, while the `getOrder()` method defines the order in which the filter is executed relative to other filters.

Here's an example of a custom web filter:

```
@Component

public class MyWebFilter implements WebFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain
chain) {

        // Pre-processing logic

        // ...

        // Pass the request to the next filter or the target handler
        return chain.filter(exchange)

            .doOnSuccess(signal -> {

                // Post-processing logic

                // ...

            });
    }
}
```

```

@Override

public int getOrder() {

    // Specify the order of the filter execution

    return Ordered.HIGHEST_PRECEDENCE;

}

}

```

In the above example, `MyWebFilter` implements the `WebFilter` interface and overrides the `filter()` method to define the filter's behavior. The `getOrder()` method returns `Ordered.HIGHEST_PRECEDENCE` to indicate that this filter should be executed first.

To register the custom filter, you can use the `@Component` annotation to mark it as a Spring bean. Spring WebFlux automatically detects and applies the filter during request processing.

Configuring Filter Ordering and Execution Chains

The order of execution of web filters can be controlled by the `getOrder()` method in the `WebFilter` interface. The lower the return value, the earlier the filter is executed. The `Ordered` interface provides constants for commonly used filter orders, such as `Ordered.HIGHEST_PRECEDENCE`, `Ordered.LOWEST_PRECEDENCE`, etc. You can also implement the `Ordered` interface or use the `@Order` annotation to specify a custom order.

In addition to controlling individual filter order, you can also configure filter execution chains by using the `WebFilterChain` interface. The `WebFilterChain` represents the chain of filters and handlers that process the request. By invoking `chain.filter(exchange)` in your custom filter, you pass the request to the next filter or the target handler in the chain.

Handling Common Scenarios with Web Filters

Web filters are versatile and can be used to handle various common scenarios, including:

- **Logging:** Create a logging filter to log request and response details, such as URL, headers, and payloads, for auditing or debugging purposes.
- **Authentication and Authorization:** Implement an authentication filter to verify user credentials, perform token validation, or enforce access control rules.
- **Caching:** Develop a caching filter to intercept responses and cache them to improve performance by serving cached responses for subsequent identical requests.
- **Request/Response Modification:** Build a filter to modify request parameters, headers, or bodies, or modify the response by adding custom headers or transforming the content.
- **Error Handling:** Create an error handling filter to intercept exceptions or errors and customize the response accordingly.

By creating custom web filters and configuring their order and execution chains, you can implement and apply cross-cutting concerns, handle common scenarios, and introduce additional functionality to your Spring WebFlux application.

It's important to note that Spring WebFlux also provides other ways to implement cross-cutting concerns, such as interceptors and aspect-oriented programming (AOP). Depending on the specific use case, you can choose the most suitable approach to achieve the desired behavior in your application.